

	WetSAT-ML Version 1.0	
	Wetlands flooding extent and trends using SAT ellite data and M achine Learning Version 1.0	

User Manual – WetSAT v1.0

Wetlands flooding extent and trends using SATellite data and Machine Learning by
Stockholm Environment Institute (SEI)

Objective

Develop a manual or user guide for the correct and efficient use of the WetSAT version 1.0 tool, through five (5) instructions or guidelines. Similarly, this manual seeks to offer technical support to users if errors, bugs, or inconsistencies are identified or arise during the execution of the Random Forest (RF) and K-means (K) models.

Introduction to WetSAT

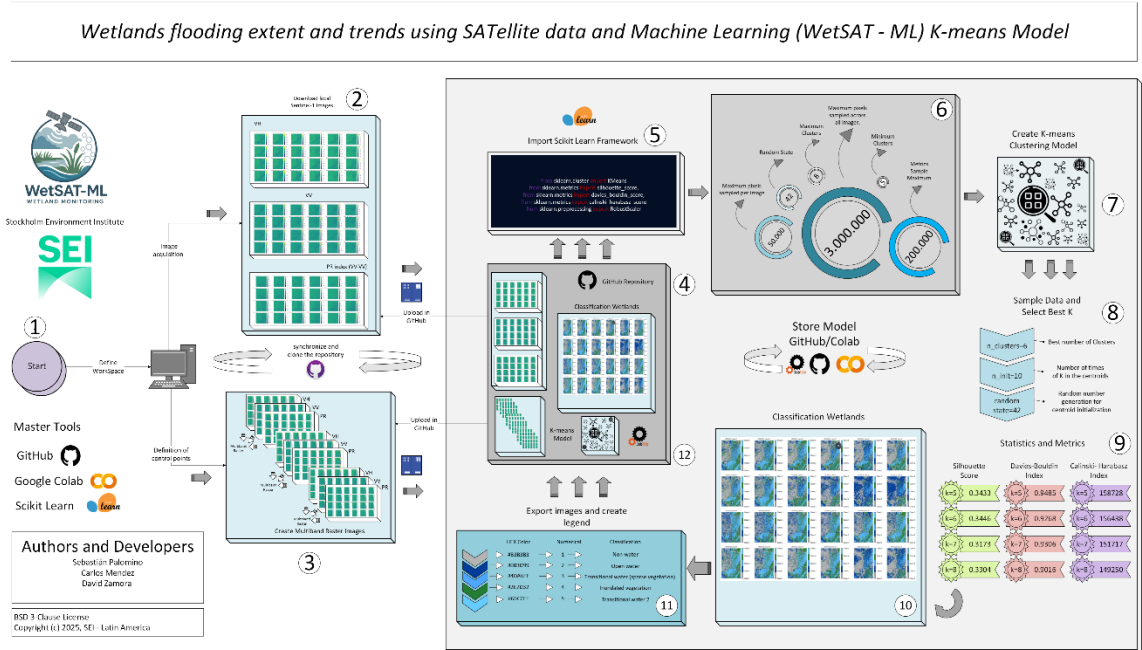
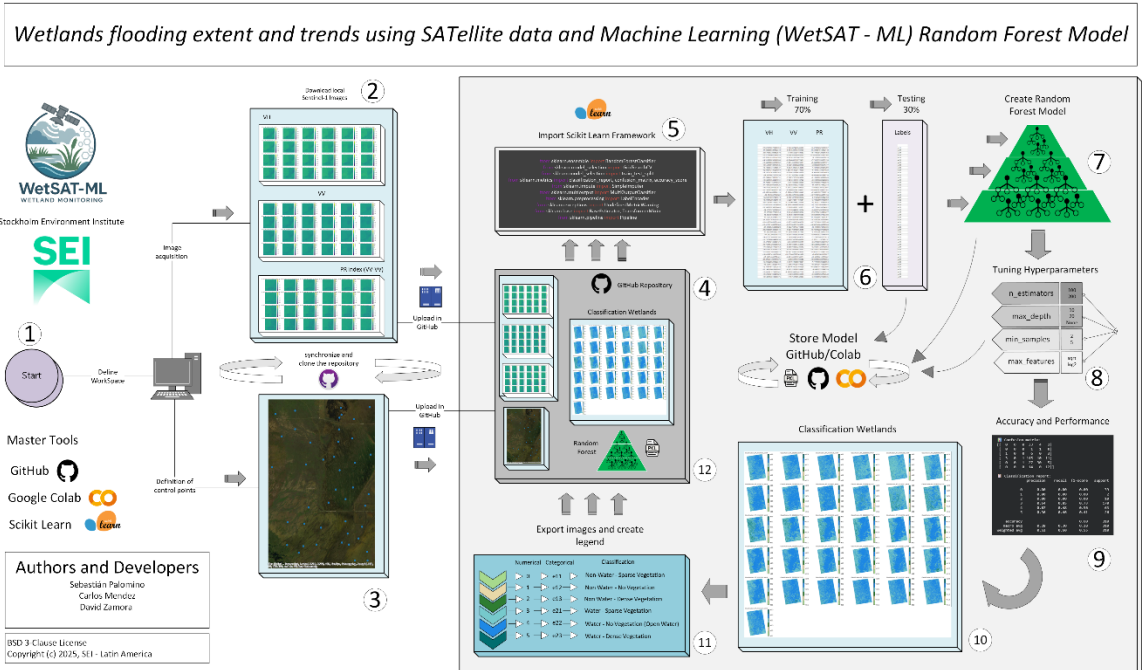
WetSAT-ML (Wetlands flooding extent and trends using SATellite data and Machine Learning) version 2.0, It consists of an open-source algorithm integrated with Google Colab and Scikit Learn. The tool processes radar satellite data from the Sentinel-1 mission to generate wetland flooding extent maps, water permanence maps, and quantify key hydrological parameters, including flooded area time series, hydroperiods, and intra- and inter-annual wetland area trends. The algorithm will use machine learning models (Random Forest and K-means) to characterize the scattering behavior of the radar signal for different wetland flooding conditions, enabling a pixel-level water detection in the satellite images.

 *Important information only for researchers, programmers,
and developers*

The SEI team for Latin America (SEI-LA) has created comprehensive and detailed documentation for the WetSAT tool on GitHub Repositories. Please note the following considerations:

1. There are two versions of the WetSAT tool code available in R and Python languages.
 - WetSAT in Python Language: [GitHub - sei-latam/WETSAT_v2: Wetlands flooding extent and trends using SATellite data and Machine Learning v2.0](#)
 - WetSAT in R Language: [GitHub - sei-latam/WETSAT: What the Package Does \(One Line, Title Case\)](#)
2. If you wish to make modifications or adjustments to the master codes of the Random Forest and K-means models, please fork or clone the repositories of Python or R.
 - Master code of Random Forest Model in Google Colab: [WETSAT_v2/2_Modelling_WETSAT_Google_Colab/Master_Random_Forest_WetSAT.ipynb at main · sei-latam/WETSAT_v2 · GitHub](#)
 - Master code of K-means Model in Google Colab: [WETSAT_v2/2_Modelling_WETSAT_Google_Colab/Master_Kmeans.ipynb at main · sei-latam/WETSAT_v2 · GitHub](#)
3. Contributions are accepted and welcome, if they are made through issues or pull requests within the repository.
 - Contributions and Pull Request: [Pull requests · sei-latam/WETSAT_v2 · GitHub](#)
4. Before making any modifications to the master codes, please check for available updates to the Scikit Learn Framework.
 - Scikit -Learn Official Repository: [scikit-learn · GitHub](#)

5. We recommend following the design and architecture of the WetSAT tool.



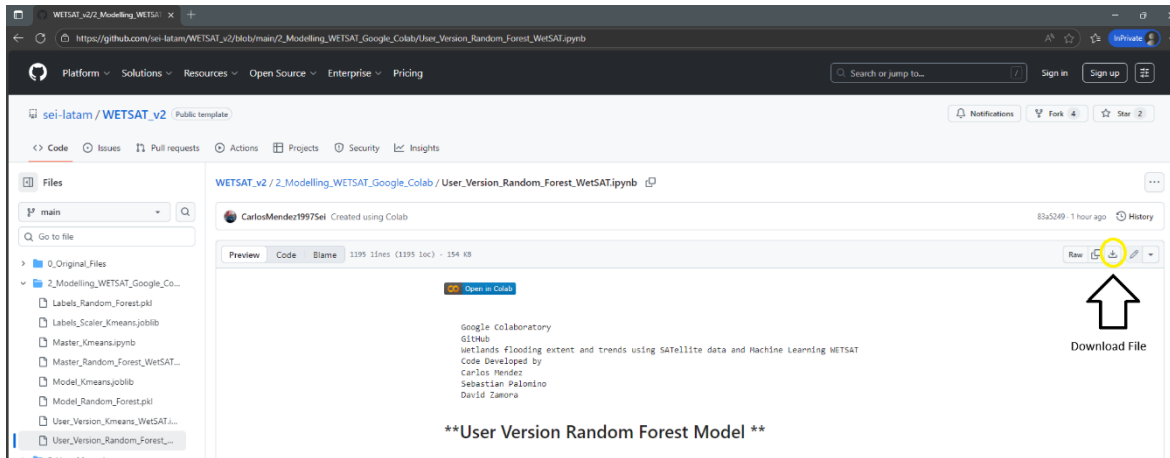
6. There may be future updates or releases of WetSAT by the SEI-LA team, so we recommend periodically synchronizing the repository.

User Manual – WetSAT v1.0

1. Prerequisites and Python files

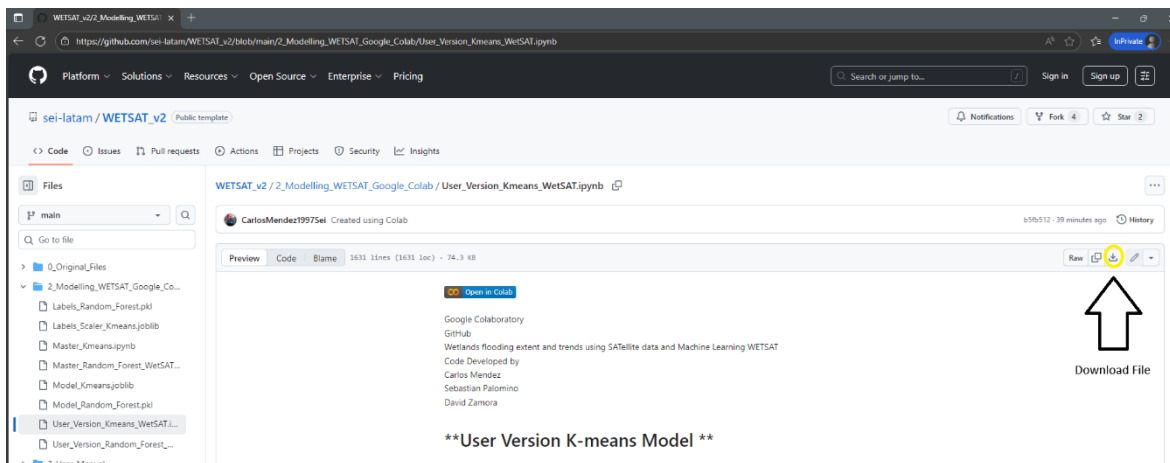
Random Forest Model

- Download  the Python 3 file with the .ipynb extension ([User_Random_Forest_WetSAT.ipynb](#)):



K-means Model

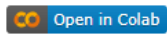
- Download  the Python 3 file with the .ipynb extension ([User_Kmeans_WetSAT.ipynb](#)):



2. Configure Workspace and Dependences

Open or import the Python 3 files downloaded previously (User_Random_Forest_WetSAT.ipynb or User_Kmeans_WetSAT.ipynb) in your preferred Integrated Development Environment (IDE) (Google Colab, PyCharm, Visual Studio Code, Jupyter Notebook, etc.) and install the libraries and packages used in WetSAT.

If you want to use Google Colab, please click the button labeled “Open in Colab”



or open the following URLs and then login or signup:

Random Forest:

[User_Version_Random_Forest_WetSAT.ipynb - Colab](#)

K-means:

[User_Version_Kmeans_WetSAT.ipynb - Colab](#)

2.1 Install Libraries and Packages used in WetSAT:

By default, all libraries and packages used in WetSAT are encoded in Google Colab scripts. The libraries and packages, mainly of the Scikit-Learn Framework are briefly described below.

2.1.1 Libraries and Packages of Random Forest

```
##### Artificial Intelligence Frameworks
# scikit-learn Framework
!pip install scikit-learn
##### Data, Geoprocessing and Graphics libraries
!pip install rasterio
!pip install matplotlib
!pip install numpy

## AI packages
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.impute import SimpleImputer
from sklearn.multioutput import MultiOutputClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.exceptions import UndefinedMetricWarning
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline

## Geoprocessing packages
import rasterio
from rasterio.features import rasterize
from matplotlib.colors import ListedColormap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import matplotlib.colors as mcolors
import gc # Garbage collector to free memory
import os
import glob
import joblib
import urllib.request
import warnings
from collections import Counter
from google.colab import files
from google.colab import data_table
```

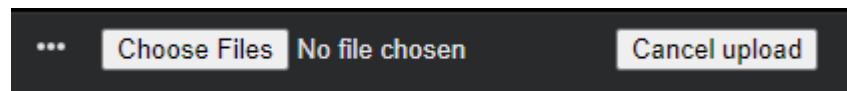
2.1.2 Libraries and Packages of K-means

```
##### Artificial Intelligence Frameworks
# scikit-learn Framework
!pip install scikit-learn
##### Data, Geoprocessing and Graphics libraries
!pip install rasterio
!pip install matplotlib
!pip install numpy
!pip install contextily
!pip install pandas
## AI packages
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
from sklearn.preprocessing import RobustScaler
## Processing packages
import os # Provides functions to interact with the operating system (paths, directories).
import re # Regular expressions for pattern matching filenames.
import random # Random number generation (used for sampling pixels)
import warnings # To manage warning messages.
from glob import glob # Utility to find files matching a pattern in directories.
import matplotlib.pyplot as plt
import joblib # For saving/loading Python objects (scaler, KMeans model).
import numpy as np # Numerical computing library (arrays, math).
import pandas as pd # Data manipulation library (tables, CSV export).
import rasterio # Library to read/write raster (GeoTIFF) files.
from rasterio.errors import NotGeoreferencedWarning # Specific warning type to suppress.
warnings.filterwarnings("ignore", category=NotGeoreferencedWarning)
from collections import Counter
from google.colab import files
from google.colab import data_table
import urllib.request
```

3. Prepare Inputs and Data

3.1 Import VV and VH Images

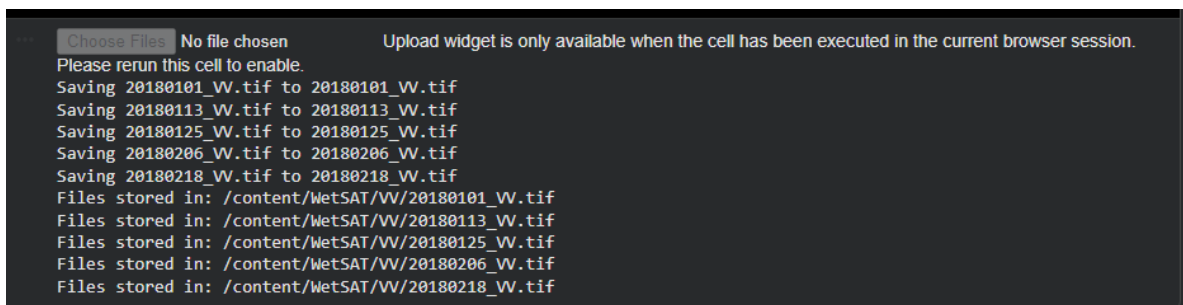
The import or upload of VH and VV images for the Random Forest and K-means models is performed manually by the user using the following button:



The user must select “Choose files” to initially upload the VH images and then the VV images. Considering that this step is performed by the user, it is recommended to follow these instructions:

1. Only upload raster images with the (.tif) or (.geotiff) extension, as the RF and K models were trained using this file extension.
2. The number of VV and VH images must be the same in size and location (area of interest).
3. There is no limit or maximum number of VH and VV images to be classified. However, if you want to perform mass or large-scale classifications, it is recommended to work in batches or small groups.
4. If you encounter any problems uploading images, we recommend canceling the process and running it again.

A successful image upload process should appear as follows:



```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session.
Please rerun this cell to enable.
Saving 20180101_VV.tif to 20180101_VV.tif
Saving 20180113_VV.tif to 20180113_VV.tif
Saving 20180125_VV.tif to 20180125_VV.tif
Saving 20180206_VV.tif to 20180206_VV.tif
Saving 20180218_VV.tif to 20180218_VV.tif
Files stored in: /content/WetSAT/VV/20180101_VV.tif
Files stored in: /content/WetSAT/VV/20180113_VV.tif
Files stored in: /content/WetSAT/VV/20180125_VV.tif
Files stored in: /content/WetSAT/VV/20180206_VV.tif
Files stored in: /content/WetSAT/VV/20180218_VV.tif

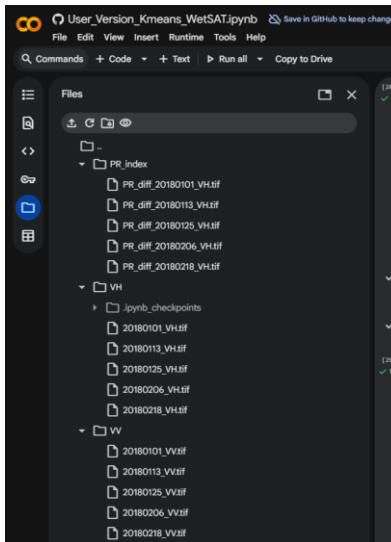
```

3.2 Creation of the dual Polarimetric Radar index (PR)

The dual Polarimetric Radar index (PR), it's a common index using in studies of remote sensing and Sentinel-1 imageries. In general way, the PR calculate the difference between the ratio of vertically transmitted and horizontally received (VH) backscatter to the vertically transmitted and vertically received (VV) backscatter. Usually, PR index provide information about surface properties (vegetation, land, water, weather, etc.).

$$\text{Polarimetric Radar index (PR)} = (VH - VV)$$

The PR index is calculated automatically once the user has uploaded the VH and VV images from the previous step. This means that the user should have three folders (VH, VV, and PR) in their work environment, as shown in the following example:



4. Run and Compile Random Forest or K-means models

In this phase, the compilation and execution of the Random Forest and K-means models were carried out in three completely different processes, due to the fact that each model has a different functioning and operation. The Random Forest model is categorized as a supervised model. Meanwhile, the K-means model is categorized as an unsupervised model. The three processes for the Random Forest and K-means models are briefly;

4.1 Run process Random Forest.

- Import the pre-trained Random Forest model from GitHub:
[WETSAT_v2/2_Modelling_WETSAT_Google_Colab/Model_Random_Forest.pkl at main · sei-latam/WETSAT_v2 · GitHub](#)
- Import the labels and marks from GitHub:
[WETSAT_v2/2_Modelling_WETSAT_Google_Colab/Labels_Random_Forest.pkl at main · sei-latam/WETSAT_v2 · GitHub](#).
- Start classifications:

The model and labels in Random Forest from GitHub allow you to access the default settings with which the model was developed during the master code. In this section, the objective is to be able to use the information stored in the pickle file (.pkl), such as the hyperparameters `{‘max_depth’: 10, ‘max_features’: ‘sqrt’, ‘min_samples_leaf’: 2, ‘min_samples_split’: 5, ‘n_estimators’: 200}` and the data distribution (70% training and

30% validation) of the original model to perform new classifications, such as in a new area of interest or images of different temporalities.

The expected results of the random forest model include multiband raster images, composed from the stacking of the three images (VH, VV, and PR) with classification according to the six categories proposed by the SEI Latin America research team.

```
'e11': '#A7D98D', # Non-water - Sparse vegetation
'e12': '#E8D8A8', # Non-water - No vegetation
'e13': '#2E7D32', # Non-water - Dense vegetation
'e21': '#76C9CC', # Water - Sparse vegetation
'e22': '#1E88E5', # Water - No vegetation (open water)
'e23': '#006D6F', # Water - Dense vegetation
```

4.2 Run process K-means.

- Import the pre-trained K-means model from GitHub:
[WETSAT_v2/2_Modelling_WETSAT_Google_Colab/Model_Kmeans.joblib at main · sei-latam/WETSAT_v2 · GitHub](#)
- Import the scalers from GitHub:
[WETSAT_v2/2_Modelling_WETSAT_Google_Colab/Labels_Scaler_Kmeans.joblib at main · sei-latam/WETSAT_v2 · GitHub](#)
- Start classifications:

The K-means model and scalers imported from GitHub, as in the Random Forest model, allow access to the default configuration with which the model was developed during the master code, which is stored in a joblib file (.joblib) with important information {Maximum of 50,000 pixels sampled per image, maximum of 1,000,000 to 3,000,000 pixels sampled in all images, K-minimum of 5, K-maximum of 8, random state of 42, and maximum sample of 200,000 metrics}. Likewise, a number of clusters of 6.

The expected results of the K-means model include multiband raster images, composed of the stacking of the three images (VH, VV, and PR) with classification according to the five categories proposed by the SEI Latin America research team.

```
1: "Non-water", #B3B3B3
2: "Open water", #003D99
3: "Transitional", #4DA6FF
4: "Inundated vegetation", #2E7D32
5: "Transitional water 2", #66CCFF
```

5. Classification maps, creating reports and export data

In this final phase, two processes and analyses are carried out, which provide the results of WetSAT. The final Random Forest and K-means classification processes are described below, along with their reports and data.

5.1 Reports and Data

It is possible to create a report of the areas in square kilometers (km²) classified according to the model used (6 categories for Random Forest and 5 categories for K-means). Therefore, tables and reports in csv are created with the names of the .tif raster images, the code or label, the number of pixels classified per raster image, and the total area occupied in the image by each RF or K-means category.

Random Forest Model

```
***
  archivo      code etiqueta  pixeles  area_km2
0 Classification_rf_gamma_0_20240101_VV_sample.tif 0 c11 2631 0 1.0524
1 Classification_rf_gamma_0_20240101_VV_sample.tif 2 c13 6273 1 2.5002
2 Classification_rf_gamma_0_20240101_VV_sample.tif 3 c21 741615 2 296.6460
3 Classification_rf_gamma_0_20240101_VV_sample.tif 4 c22 188895 3 43.5580
4 Classification_rf_gamma_0_20240101_VV_sample.tif 5 c23 234593 4 89.4372
5 Classification_rf_gamma_0_20240111_VV_sample.tif 0 c11 2840 5 1.1360
6 Classification_rf_gamma_0_20240111_VV_sample.tif 2 c13 7147 0 2.8540
7 Classification_rf_gamma_0_20240111_VV_sample.tif 3 c21 733409 7 291.2436
8 Classification_rf_gamma_0_20240111_VV_sample.tif 4 c22 88811 8 35.5244
9 Classification_rf_gamma_0_20240111_VV_sample.tif 5 c23 252100 9 100.0400
10 Classification_rf_gamma_0_20240125_VV_sample.tif 0 c11 7796 10 1.1104
11 Classification_rf_gamma_0_20240125_VV_sample.tif 1 c12 1 13 0.0004
12 Classification_rf_gamma_0_20240125_VV_sample.tif 2 c13 6865 12 2.6364
13 Classification_rf_gamma_0_20240125_VV_sample.tif 3 c21 724618 13 289.8440
14 Classification_rf_gamma_0_20240125_VV_sample.tif 4 c22 38191 14 12.0764
15 Classification_rf_gamma_0_20240125_VV_sample.tif 5 c23 538123 15 128.1275
16 Classification_rf_gamma_0_20240206_VV_sample.tif 0 c11 2846 16 1.1385
17 Classification_rf_gamma_0_20240206_VV_sample.tif 2 c13 6568 17 2.5272
18 Classification_rf_gamma_0_20240206_VV_sample.tif 3 c21 719493 18 287.6132
19 Classification_rf_gamma_0_20240206_VV_sample.tif 4 c22 28165 19 8.8660
20 Classification_rf_gamma_0_20240206_VV_sample.tif 5 c23 235395 20 131.1500
21 Classification_rf_gamma_0_20240218_VV_sample.tif 0 c11 3746 21 1.4984
22 Classification_rf_gamma_0_20240218_VV_sample.tif 1 c12 1 22 0.0004
23 Classification_rf_gamma_0_20240218_VV_sample.tif 2 c13 7495 23 2.9000
24 Classification_rf_gamma_0_20240218_VV_sample.tif 3 c21 731611 24 291.4444
25 Classification_rf_gamma_0_20240218_VV_sample.tif 4 c22 31806 25 12.4444
26 Classification_rf_gamma_0_20240218_VV_sample.tif 5 c23 808868 26 123.2272
CSV file exported as "area_by_class.csv"
```

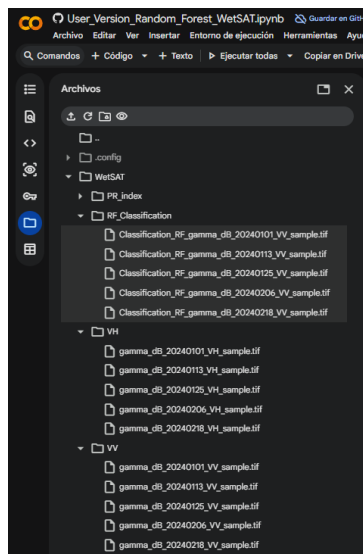
K-means Model

```
***
  archivo      code etiqueta  pixeles  area_km2
0 20240101_class.tif 1 Non-water 4541 1.8164
1 20240101_class.tif 2 Open water 73853 29.5412
2 20240101_class.tif 3 Transitional 279340 111.7360
3 20240101_class.tif 4 Inundated vegetation 568378 227.3512
4 20240101_class.tif 5 Transitional water 2 154842 61.9368
5 20240113_class.tif 1 Non-water 3761 1.5044
6 20240113_class.tif 2 Open water 48897 19.5588
7 20240113_class.tif 3 Transitional 297057 118.8228
8 20240113_class.tif 4 Inundated vegetation 596188 238.4752
9 20240113_class.tif 5 Transitional water 2 135051 54.0204
10 20240125_class.tif 1 Non-water 6971 2.7884
11 20240125_class.tif 2 Open water 4850 1.9400
12 20240125_class.tif 3 Transitional 298195 119.2780
13 20240125_class.tif 4 Inundated vegetation 715289 286.1156
14 20240125_class.tif 5 Transitional water 2 55649 22.2596
15 20240206_class.tif 1 Non-water 7761 3.1044
16 20240206_class.tif 2 Open water 1377 0.5508
17 20240206_class.tif 3 Transitional 291921 116.7684
18 20240206_class.tif 4 Inundated vegetation 752660 301.0640
19 20240206_class.tif 5 Transitional water 2 27235 10.8940
20 20240218_class.tif 1 Non-water 4162 1.6648
21 20240218_class.tif 2 Open water 6969 2.7876
22 20240218_class.tif 3 Transitional 297952 119.1728
23 20240218_class.tif 4 Inundated vegetation 708878 280.1512
24 20240218_class.tif 5 Transitional water 2 71013 28.4052
CSV file exported as "area_by_class.csv"
```

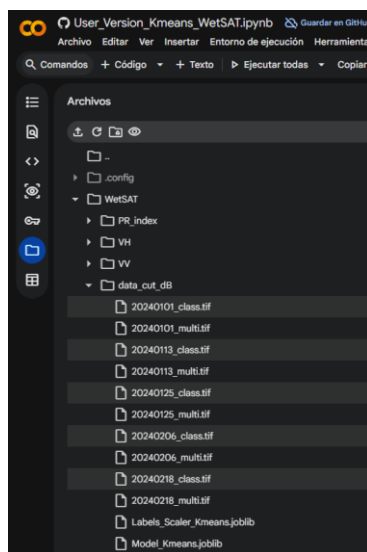
5.2 Final Classifications

The final classification and visualization of the images used by the user with WetSAT requires the correct execution of phases three (Prepare Inputs and Data) and four (Run and Compile Random Forest or K-means models). In general terms, once the user has successfully imported their VH and VV images and selected the classification model (Random Forest or K-means), The code developed will use this information to create new (.tif) raster images with the RF or K- categories that the system has previously identified.

Example to download raster images from Workspace Random Forest

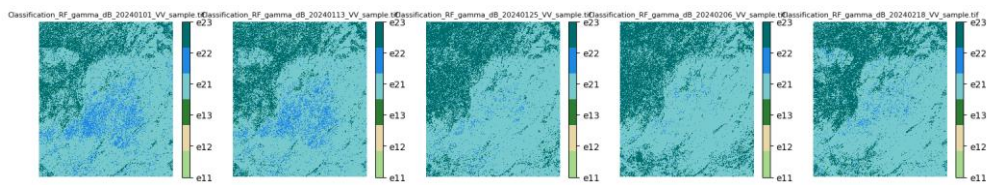


Example to download raster images from Workspace K-means



Finally, once the user's new images have been classified with the RF or K- models, it is possible to download these new images into the workspace or create a preview of the images, as shown below.

Random Forest Model



K-means Model

